

Practical Work 3: MPI File Transfer

Student Name

1 Objective

The objective of this practical work is to upgrade the file transfer system into a distributed file transfer system using the Message Passing Interface (MPI). Two distributed processes exchange files using message-based communication.

2 Choice of MPI Implementation

MPICH was selected as the MPI implementation because it is open-source, easy to install on both Windows and Linux, well documented, and widely used in academic environments. It also fully supports the standard MPI communication primitives.

3 MPI Service Design

The MPI service consists of two processes. Process 0 acts as the sender and Process 1 acts as the receiver. The sender transmits the file size followed by the file data.

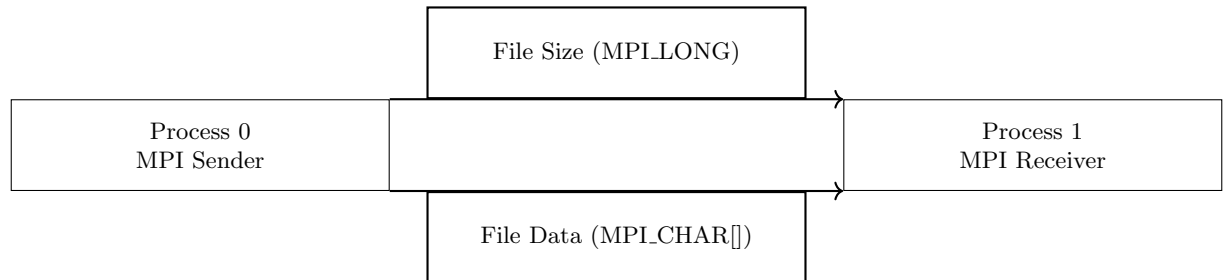


Figure 1: MPI Service Design for File Transfer

4 System Organization

The system is executed using the `mpiexec` command which launches two MPI processes simultaneously. One process sends the file and the other receives it and stores it locally.

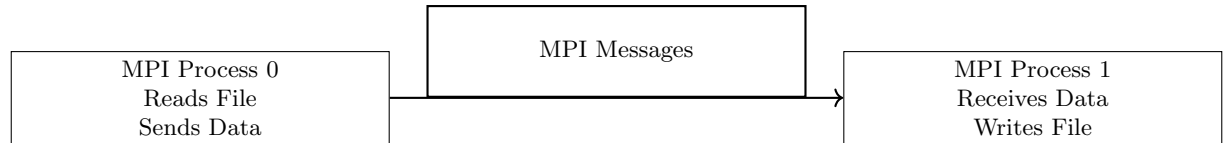


Figure 2: System Organization of MPI File Transfer

5 Implementation

5.1 Sender Code (mpi_sender.c)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        FILE *fp = fopen("test.txt", "rb");
        if (!fp) {
            printf("Cannot open file\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }

        fseek(fp, 0, SEEK_END);
        long size = ftell(fp);
        rewind(fp);

        char *buffer = (char*) malloc(size);
        fread(buffer, 1, size, fp);

        MPI_Send(&size, 1, MPI_LONG, 1, 0, MPI_COMM_WORLD);
        MPI_Send(buffer, size, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
    }
}
```

```

        printf("File sent via MPI (%ld bytes)\n", size);

        fclose(fp);
        free(buffer);
    }

    MPI_Finalize();
    return 0;
}

```

5.2 Receiver Code (mpi_receiver.c)

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 1) {
        long size;
        MPI_Recv(&size, 1, MPI_LONG, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        char *buffer = (char*) malloc(size);
        MPI_Recv(buffer, size, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        FILE *fp = fopen("received_mpi.txt", "wb");
        fwrite(buffer, 1, size, fp);

        printf("File received via MPI (%ld bytes)\n", size);

        fclose(fp);
        free(buffer);
    }

    MPI_Finalize();
    return 0;
}

```

6 Role Distribution

Process 0 is responsible for reading the file and sending its content. Process 1 is responsible for receiving the file and saving it to disk. The MPI communication layer ensures data transmission and synchronization.

7 Conclusion

This project demonstrates how MPI can be used to implement a distributed file transfer system using message passing. Compared to socket-based TCP and RPC-based methods, MPI provides direct process-level communication and parallel execution.